```
pos = Order.where("name = 'Dave' and pay_type = 'po'")
```

The result will be an `ActiveRecord::Relation` object containing all the matching rows, each neatly wrapped in an `Order` object.

That's fine if our condition is predefined, but how do we handle it when the name of the customer is set externally (perhaps coming from a web form)? One way is to substitute the value of that variable into the condition string:

```
# get the name from the form
name = params[:name]
# DON'T DO THIS!!!
pos = Order.where("name = '#{name}' and pay_type = 'po'")
```

As the comment suggests, this isn't a good idea. Why? It leaves the database wide open to something called a *SQL injection* attack, which the Ruby on Rails Guides[99] describe in more detail. For now, take it as a given that substituting a string from an external source into a SQL statement is effectively the same as publishing your entire database to the whole online world.

Instead, the safe way to generate dynamic SQL is to let Active Record handle it. Doing this allows Active Record to create properly escaped SQL, which is immune from SQL injection attacks. Let's see how this works.

If we pass multiple parameters to a `where` call, Rails treats the first parameter as a template for the SQL to generate. Within this SQL, we can embed placeholders, which will be replaced at runtime by the values in the rest of the array.

One way of specifying placeholders is to insert one or more question marks in the SQL. The first question mark is replaced by the second element of the array, the next question mark by the third, and so on. For example, we could rewrite the previous query as this:

```
name = params[:name]
pos = Order.where(["name = ? and pay_type = 'po'", name])
```

We can also use named placeholders. We do that by placing placeholders of the form `:name` into the string and by providing corresponding values in a hash,